

竞赛打印版

1. 赛场策略

做题节奏

- 开场先快速扫题，找签到题和中档题。
- 一题卡 20 到 30 分钟还没有明确做法，先换题。
- 优先拿稳分，不要一开始就在第一题耗太久。
- 每过一题都回头看一眼边界和输出格式再交。

读题顺序

- 先看数据范围。
- 再看输入输出格式。
- 再看是否多组数据。
- 再看下标从 0 还是 1 开始。
- 再看样例在提示什么性质。

卡题自救

- 先分清卡在思路、证明、实现还是复杂度。
- 先写暴力，确认题意和性质，再考虑优化。
- 多手推小样例，优先看 $n=1$ 、全相同、全负数、无解。
- 做法太复杂时，优先怀疑方向错了。

提交前检查

- 是否清空了上组数据。
- 是否漏了特判。
- 是否越界。
- 是否 `int` 爆了，需要 `long long`。
- 是否有除零、负取模、空容器访问问题。
- 输出格式是否严格符合题意。

2. 基础模板

ACM 基础模板

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T = 1;
    // cin >> T;
    while (T--) solve();
    return 0;
}
```

常用输入

```
int n;
cin >> n;

int a, b;
cin >> a >> b;

vector<int> v(n);
for (int i = 0; i < n; i++) cin >> v[i];
```

多组数据

```
int T;
cin >> T;
while (T--) solve();
```

读一整行

```
string s;
getline(cin, s);
```

如果前面用了 `cin >>`，要先吃掉换行：

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
getline(cin, s);
```

读到文件结束

```
int x;
while (cin >> x) {
}
```

读二维数组

```
int n, m;
cin >> n >> m;
vector<vector<int>> a(n, vector<int>(m));
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        cin >> a[i][j];
    }
}
```

读字符矩阵

```
int n, m;
cin >> n >> m;
vector<string> g(n);
for (int i = 0; i < n; i++) cin >> g[i];
```

读图

```
int n, m;
cin >> n >> m;
vector<vector<int>> g(n + 1);
for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
}
```

读带权图

```
int n, m;
cin >> n >> m;
vector<vector<pair<int, int>>> g(n + 1);
for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    g[u].push_back({v, w});
    g[v].push_back({u, w});
}
```

输出数组

```
for (int i = 0; i < n; i++) {
    cout << a[i] << " \n"[i == n - 1];
}
```

输出精度

```
cout << fixed << setprecision(6) << x << '\n';
```

3. 头文件速查

常用头文件

- `#include <iostream>`: `cin`、`cout`
- `#include <vector>`: `vector`
- `#include <string>`: `string`
- `#include <algorithm>`: `sort`、`reverse`、`lower_bound`
- `#include <queue>`: `queue`、`priority_queue`
- `#include <stack>`: `stack`
- `#include <set>`: `set`、`multiset`
- `#include <map>`: `map`
- `#include <unordered_map>`: `unordered_map`
- `#include <unordered_set>`: `unordered_set`
- `#include <deque>`: `deque`
- `#include <cmath>`: `sqrt`、`pow`
- `#include <cstring>`: `memset`
- `#include <numeric>`: `gcd`、`lcm`、`accumulate`、`iota`

- `#include <iomanip>: setprecision`
- `#include <sstream>: stringstream`
- `#include <bitset>: bitset`
- `#include <cctype>: isdigit、 isalpha`
- `#include <functional>: greater<>`
- `#include <utility>: pair`

bits/stdc++.h

- `#include <bits/stdc++.h>` 是 GNU 扩展，不是标准 C++ 头文件。
- GCC 环境通常能用。
- 如果比赛环境不确定，用标准头文件更稳。

4. 常用数据结构

vector

- 头文件： `<vector>`
- 作用：动态数组

```
vector<int> a;  
a.push_back(x);  
a.pop_back();  
a.size();  
a.empty();  
a.clear();  
a.front();  
a.back();  
a[i];  
sort(a.begin(), a.end());  
reverse(a.begin(), a.end());
```

string

- 头文件： `<string>`
- 作用：字符串

```
string s;  
s.size();  
s.empty();  
s.clear();  
s.push_back('a');  
s.pop_back();  
s.substr(pos, len);  
s.find("abc");  
s.erase(pos, len);  
s.insert(pos, "abc");  
s.front();  
s.back();
```

pair

- 头文件: <utility>

```
pair<int, int> p = {1, 2};  
cout << p.first << ' ' << p.second << '\n';
```

queue

- 头文件: <queue>

```
queue<int> q;  
q.push(x);  
q.pop();  
q.front();  
q.back();  
q.size();  
q.empty();
```

stack

- 头文件: <stack>

```
stack<int> st;  
st.push(x);  
st.pop();  
st.top();  
st.size();  
st.empty();
```

deque 是双端队列

- 头文件: <deque>

```
deque<int> dq;  
dq.push_back(x);  
dq.push_front(x);  
dq.pop_back();  
dq.pop_front();  
dq.front();  
dq.back();
```

priority_queue

- 头文件: <queue>
- 默认大根堆

```
priority_queue<int> q1;  
priority_queue<int, vector<int>, greater<int>> q2;  
q1.push(x);  
q1.top();  
q1.pop();
```

set / multiset

- 头文件: <set>
- set 去重且有序, multiset 可重复

```
set<int> s;  
s.insert(x);  
s.erase(x);  
s.count(x);  
s.find(x);  
s.lower_bound(x);  
s.upper_bound(x);
```

map

- 头文件: <map>
- 按 key 自动排序

```
map<string, int> mp;  
mp["apple"] = 1;  
mp.count("apple");  
mp.find("apple");  
mp.erase("apple");
```

注意:

- mp[key] 如果 key 不存在, 会自动创建。
- find 返回迭代器, 不是下标。

unordered_map / unordered_set

- 头文件: <unordered_map>、<unordered_set>
- 哈希结构, 无序, 平均更快

```
unordered_map<int, int> mp;  
unordered_set<int> s;
```

bitset

- 头文件: <bitset>

```
bitset<1000> bs;  
bs.set();  
bs.reset();  
bs.flip();  
bs.count();  
bs.any();  
bs.none();  
bs[i];
```

5. STL 高频函数

排序

```
sort(a.begin(), a.end());  
sort(a.begin(), a.end(), greater<int>());
```

如果不用 `greater<int>()`，也可以自己写比较函数：

```
bool cmp(int a, int b) {  
    return a > b; // 降序  
}  
sort(a.begin(), a.end(), cmp);
```

自定义排序

```
bool cmp(const pair<int, int>& a, const pair<int, int>& b) {  
    return a.second < b.second;  
}  
sort(v.begin(), v.end(), cmp);
```

规则：

- `sort` 的比较函数返回 `true`，表示 `a` 应该排在 `b` 前面。
- 想升序就写 `a < b`，想降序就写 `a > b`。
- 比较函数参数尽量写成 `const T&`，避免拷贝。

也可以用 `lambda`：

```
sort(v.begin(), v.end(), [](const pair<int, int>& a, const pair<int, int>& b) {  
    return a.second < b.second;  
}));
```

priority_queue 自定义比较器

priority_queue<int, vector<int>, greater<int>> 里的 greater<int> 也可以换成你自己写的比较器。

写法一：结构体

```
struct Cmp {
    bool operator()(int a, int b) const {
        return a > b; // 小根堆
    }
};
priority_queue<int, vector<int>, Cmp> pq;
```

写法二：比较 pair

```
struct CmpPair {
    bool operator()(const pair<int, int>& a, const pair<int, int>& b) const {
        return a.second > b.second; // second 小的优先
    }
};
priority_queue<pair<int, int>, vector<pair<int, int>>, CmpPair> pq;
```

注意：

- sort 和 priority_queue 的比较器理解方式容易反。
- 对 sort 来说，返回 true 表示 a 排在前面。
- 对 priority_queue 来说，比较器决定“谁优先级更低”，默认写法更容易写反，建议直接记模板。

二分

```
lower_bound(a.begin(), a.end(), x);
upper_bound(a.begin(), a.end(), x);
```

累加

```
int sum = accumulate(a.begin(), a.end(), 0);
```

填充连续值

```
iota(a.begin(), a.end(), 0);
```

反转

```
reverse(a.begin(), a.end());
```

去重

```
sort(a.begin(), a.end());  
a.erase(unique(a.begin(), a.end()), a.end());
```

统计

```
count(a.begin(), a.end(), x);  
max_element(a.begin(), a.end());  
min_element(a.begin(), a.end());
```

6. 数学与位运算

gcd

```
int gcd(int a, int b) {  
    while (b) {  
        int t = b;  
        b = a % b;  
        a = t;  
    }  
    return a;  
}
```

或直接用：

```
std::gcd(a, b);
```

exgcd

```
int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}
```

快速幂

```
long long qpow(long long a, long long b, long long mod) {
    long long res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

快速乘

```
long long qmul(long long a, long long b, long long mod) {
    long long res = 0;
    while (b) {
        if (b & 1) res = (res + a) % mod;
        a = (a + a) % mod;
        b >>= 1;
    }
    return res;
}
```

位运算

- 左移: <<
- 右移: >>
- 与: &

- 或： $|$
- 异或： $\wedge$
- 取反： $\sim$

Brian Kernighan 算法

```
int cnt = 0;
while (x) {
    x &= (x - 1);
    cnt++;
}
```

7. 前缀和、差分、双指针

前缀和

```
vector<long long> sum(n + 1, 0);
for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + a[i];

long long rangeSum = sum[r] - sum[l - 1];
```

性质：

- 可用于区间和。
- 可扩展到异或前缀。
- 不能直接解决任意区间最大值。

环形前缀和

```
for (int i = 0; i < n; i++) sum[i + 1] = sum[i] + dist[i];
for (int i = n; i < 2 * n; i++) sum[i + 1] = sum[i] + dist[i - n];
```

差分

```
vector<long long> d(n + 2, 0);
d[1] += x;
d[r + 1] -= x;

for (int i = 1; i <= n; i++) a[i] = a[i - 1] + d[i];
```

双指针

常用场景：

- 滑动窗口
- 有序数组找和
- 子序列匹配

three sum

```
sort(a.begin(), a.end());
for (int i = 0; i < n; i++) {
    int l = i + 1, r = n - 1;
    while (l < r) {
        long long s = 1LL * a[i] + a[l] + a[r];
        if (s == target) {
            l++;
            r--;
        } else if (s < target) {
            l++;
        } else {
            r--;
        }
    }
}
```

8. 搜索、递归、字符串

递归 / 回溯

- 递归：函数直接或间接调用自己。
- 回溯：一条路走不通就撤销状态换路。

KMP

- 核心：求模式串每个位置之前的最长相同前后缀。
- 用于字符串匹配。
- 重点是理解 next / 前缀函数。

常用字符函数

头文件：#include <cctype>

```
isdigit(c);
isalpha(c);
isalnum(c);
isspace(c);
islower(c);
isupper(c);
tolower(c);
toupper(c);
```

字符串和数字转换

```
string s = to_string(x);
int a = stoi(s);
long long b = stoll(s);
```

stringstream

```
string line;
getline(cin, line);
stringstream ss(line);
int x;
while (ss >> x) {
}
```

9. 常见思想

区间合并

- 先按左端点排序。
- 当前区间和上一个有交就合并。

中位数 / 加权中位数

- 普通中位数常用于绝对值距离最小化。
- 加权绝对距离最小值常出现在加权中位数。

贪心

- 先找局部最优选择是否可以推出全局最优。
- 常见于排序后处理、区间选择、贡献统计。

高斯消元

- 适合线性方程组。
- 核心过程：选主元、消元、回代。

10. 易错点

输入输出

- `cin >> s` 读不到空格。
- `getline` 前面如果有 `cin >>`，要处理残留换行。
- 多组数据别忘记初始化。

类型

- $1e9 * 1e9$ 一定爆 `int`。
- 乘法、前缀和、答案统计优先考虑 `long long`。

容器

- `queue` 没有 `top()`。
- `stack` 没有 `front()`。
- `priority_queue` 默认大根堆。
- `vector::erase()` 删除中间元素代价高。

下标

- 统一使用 `0` 下标或 `1` 下标，不要混写。
- 图和树通常更适合 `1` 下标。

二分

- 先想清楚查的是第一个满足条件的位置还是最后一个满足条件的位置。
- `mid` 推荐写成：

```
int mid = l + (r - l) / 2;
```

11. 赛前速看

- 输入先读对。
- 边界先想对。
- 会做的题先拿下。
- 卡住及时换题。
- 少写花哨写法，优先写最稳的写法。